

JAVASCRIPT AVANZADO II

Resumen — memoria, npm, transpiladores y RxJS

Resumen esquemático

Basado en el fastbook original de Robert Da Corte (Frontend 02, lecciones 2 a 5 de 6)

Contenido

1. Memoria y optimización de rendimiento	3
2. Npm	3
3. Transpiladores	3
4. RxJS	3

1. Memoria y optimización de rendimiento

- ▶ JavaScript usa un **garbage collector** ("mark-and-sweep") que libera automáticamente la memoria que ya no está en uso.
- ▶ Un **memory leak** ocurre cuando objetos innecesarios siguen referenciados en algún punto del código.
- ▶ Causas comunes: variables globales innecesarias, event listeners no eliminados, referencias circulares, closures mal gestionados y timers/intervals sin limpiar.
- ▶ Herramientas de análisis: DevTools de Chrome (pestaña Memory), `node --inspect` y Lighthouse.

2. Npm

- ▶ Node.js incorpora **npm**, el gestor de paquetes de JavaScript que permite instalar, compartir y gestionar código.
- ▶ `npm init` crea el archivo **package.json**, que define dependencias, scripts e información del proyecto.
- ▶ `npm install <paquete>` añade una dependencia y crea la carpeta **node_modules**.

3. Transpiladores

- ▶ Un transpilador convierte código de un lenguaje/sintaxis a otro; en JavaScript suele convertir ES6+ a una versión más antigua (ES5) compatible con más navegadores.
- ▶ **Babel** es el transpilador más popular; se configura con `@babel/core`, `@babel/cli` y `@babel/preset-env`, junto a un archivo `.babelrc`.

4. RxJS

- ▶ **RxJS** es una biblioteca de programación reactiva basada en **observables**, muy usada junto a Angular.
- ▶ **Observable** (emite valores), **Observer** (callbacks next/error/complete), **Subscription**, **Operators** (map, filter, merge, concat), **Subject** (multicast) y **Schedulers** (controlan la concurrencia).
- ▶ Es fundamental cerrar las suscripciones para evitar memory leaks.

★ Ideas clave para recordar

- ▶ Evitar memory leaks exige cuidado con listeners, timers y referencias globales.
- ▶ npm, Babel y RxJS son piezas del ecosistema que permiten trabajar con dependencias, compatibilidad entre navegadores y programación reactiva.
- ▶ Un Observable es unicast (cada suscriptor dispara su propia ejecución); un Subject es multicast (todos comparten la misma emisión).

Viene después de "JavaScript Avanzado I": prototipos y herencia prototipal